

HOTEL MANAGEMENT SYSTEM

A Comprehensive Technical Project Report

Submitted in partial fulfillment of the requirements for the degree of

B.E. Computer Science and Business Systems

By

Janiish



DHAANISH AHMED COLLEGE OF ENGINEERING

Department of Computer Science and Business Systems

2026

DECLARATION

I, **Janiish**, a student of **B.E. Computer Science and Business Systems** at **Dhaanish Ahmed College of Engineering**, hereby declare that the project report entitled "**HOTEL MANAGEMENT SYSTEM**" submitted by me is an original work undertaken for the fulfillment of my academic curriculum.

I further declare that this project, or any part of it, has not been submitted to any other University or Institution for the award of any degree or diploma. The technical implementations, code logic, and architectural designs presented herein are the result of my independent research and developmental efforts, under the guidance of my college faculty.

Signature of the Candidate

(Janiish)

Date: _____

ACKNOWLEDGEMENT

I would like to express my deepest gratitude to all those who provided me the possibility to complete this report.

First and foremost, I offer my sincere thanks to the Head of the Department, Computer Science and Business Systems at **Dhaanish Ahmed College of Engineering**, for providing the necessary infrastructure and a conducive environment to carry out this project successfully.

I am profoundly grateful to my project guide and faculty members for their expert guidance, constructive criticism, and continuous encouragement throughout the development lifecycle of this software.

I also wish to extend my heartfelt thanks to my parents for their unwavering moral and financial support, which has been my source of inspiration.

Furthermore, this journey would not have been as enriching without the collaborative spirit and engaging discussions with my classmates and friends. A special note of thanks to **Syed, Jai Mukesh, Adil Afreed, Thawfeek Ali, and Sai Sanjay** for their constant motivation, valuable feedback, and camaraderie during our academic sessions.

Janiish

ABSTRACT

The hospitality industry is one of the fastest-growing sectors globally, demanding high efficiency, accuracy, and customer satisfaction. The Hotel Management System (HMS) presented in this report is a comprehensive, automated software application designed to streamline the intricate day-to-day operations of a modern hotel, boutique resort, or guesthouse. This project successfully integrates a robust, scalable backend developed in Java (`HotelManagement.java`) with a highly responsive, user-friendly frontend web interface designed using HTML5, CSS3, and JavaScript (`hotel.html`). Traditionally, hotel operations such as room allocation, guest registration, tariff calculation, and billing were handled manually through paper ledgers or disparate spreadsheet files. These archaic methods are heavily prone to human error, data redundancy, security breaches, and inefficient time management, directly impacting guest satisfaction and operational revenue. The proposed digital solution eradicates these bottlenecks by providing a centralized platform. The system meticulously manages room reservations in real-time, preventing double-bookings and schedule conflicts. It securely records customer demographic details, processes complex billing algorithms inclusive of dynamic taxation and auxiliary services, and facilitates seamless staff administration. This extensively detailed report documents the entire software development lifecycle (SDLC) of the HMS. It covers the preliminary feasibility study, meticulous system requirements specification (SRS), advanced architectural design utilizing Object-Oriented paradigms, practical implementation strategies, and rigorous software testing protocols. By replacing manual record-keeping with an optimized digital architecture, this project not only reduces administrative overhead but also provides a foundational prototype that can be scaled with relational databases and cloud deployment in future iterations.

CHAPTER 1: INTRODUCTION

1.1 Background and Overview

The advent of information technology has revolutionized various sectors, and the hospitality industry is no exception. A Hotel Management System is an integrated software solution that covers all aspects of hotel operations. In the modern era, where customer expectations are at an all-time high, relying on manual processes is no longer viable for any commercial accommodation provider. Fast, accurate, and seamless service is the cornerstone of a successful hotel business. This project aims to develop a robust system that handles the primary functions of a hotel front desk and back office. When a guest arrives, the system must instantly provide details on room availability across various categories (e.g., Standard, Deluxe, Suite). Upon check-in, the system must securely capture the guest's identity and contact details. Throughout the stay, any additional services requested by the guest must be logged, and upon check-out, a mathematically precise, itemized bill must be generated instantaneously.

The system is engineered to be highly scalable, ensuring that as the hotel expands its physical infrastructure (adding more rooms or amenities), the software can adapt without requiring a complete architectural overhaul. The graphical user interface (GUI) designed via HTML provides a dashboard-like experience, giving managers a bird's-eye view of current occupancy rates, daily revenue, and pending maintenance tasks.

1.2 Objectives

- To completely transition front-desk operations from manual ledgers to an automated digital database.
- To minimize human calculation errors during the complex billing and taxation process.
- To enhance the speed of the check-in and check-out processes, directly improving guest satisfaction.

- To provide a secure and centralized repository for sensitive customer information.

1.3 Problem Statement

The core motivation behind developing this software is to eliminate the inherent flaws of manual record-keeping. Manual ledgers are susceptible to physical damage, loss, and unauthorized access. Furthermore, retrieving a guest's historical data from physical files is a time-consuming process. By digitizing these records, hotel administrators can perform instant searches, generate comprehensive revenue reports, and ensure data integrity through secure digital storage. Furthermore, the project serves as a practical application of core Computer Science principles, specifically Object-Oriented Programming (OOP) in Java and modern web development practices. It demonstrates the seamless communication between a client-side interface and server-side processing logic, providing a holistic view of full-stack application development.

Therefore, the core problem is to build an application that is technically sophisticated enough to handle complex logic, yet presents an interface so simple that staff with minimal technical literacy can operate it flawlessly.

CHAPTER 2: LITERATURE REVIEW

2.1 Study of Existing Systems

Before embarking on the development of the Hotel Management System, an extensive study of existing solutions and literature was conducted to identify common pitfalls and areas for improvement.

1. **Traditional Manual Systems:** Many small-scale hotels still rely on physical registries. While cost-effective initially, this approach scales poorly. It leads to severe data redundancy, high risk of booking overlap, and significant delays during the checkout process when bills must be calculated manually.
2. **Spreadsheet-Based Management:** A step above manual registries, some establishments use software like Microsoft Excel. While this allows for easier calculation, it lacks data validation. A user can easily overwrite a cell containing a crucial formula, leading to catastrophic billing errors. Furthermore, spreadsheets do not offer a concurrent user interface, meaning multiple staff members cannot update the system simultaneously without risking data corruption.
3. **Enterprise Software (e.g., Oracle OPERA):** These are highly advanced systems used by multinational hotel chains. While they offer exhaustive features, they are prohibitively expensive for small to medium-sized hotels. They also require specialized hardware and extensive staff training.

The proposed system bridges this gap. It provides the automation and accuracy of enterprise software while maintaining the lightweight, cost-effective nature required by smaller businesses. By utilizing Java for backend logic, the system ensures platform independence and robust memory management.

2.2 Feasibility Study

Before proceeding with deep technical design, a three-pronged feasibility study was conducted:

- **Technical Feasibility:** Evaluated whether the required technologies (Java, HTML, CSS, JS) are capable of handling the system requirements. Given the robust nature of the Java Virtual Machine (JVM) and the universal compatibility of web browsers, the project is highly technically feasible.

- **Economic Feasibility:** The project utilizes open-source technologies and standard text editors/ IDEs (like Eclipse or VS Code). No proprietary software licenses are required, making the development cost negligible and highly economically feasible.
- **Operational Feasibility:** The system is designed mimicking standard hotel workflows. Receptionists will find the digital forms mirror the paper forms they are used to, ensuring high operational feasibility and low resistance to adoption.

CHAPTER 3: SYSTEM REQUIREMENTS

SPECIFICATION

3.1 Hardware Requirements

To ensure smooth deployment and operation, the following minimum hardware specifications are recommended for the host machine running the backend server:

- **Processor:** Minimum Intel Core i3 (6th Gen) or AMD Ryzen 3. Recommended: Intel Core i5 or higher for concurrent multi-user handling.
- **RAM:** Minimum 4 GB. Recommended 8 GB to provide ample heap space for the Java Virtual Machine.
- **Storage:** 256 GB SSD (Solid State Drive) is recommended for faster read/write access, though a minimum of 100 MB is required just for application binaries.
- **Display:** Standard monitor with a minimum resolution of 1366x768 pixels to ensure the HTML dashboard renders without overlapping elements.

3.2 Software Requirements

- **Operating System:** Platform independent. Capable of running on Windows (10/11), Linux (Ubuntu, Lubuntu, CentOS), or macOS.
- **Backend Environment:** Java Development Kit (JDK) Version 8, 11, or 17.
- **Frontend Environment:** Any modern web browser including Google Chrome, Mozilla Firefox, Microsoft Edge, or Apple Safari.
- **Development Tools:** VS Code, IntelliJ IDEA, or Eclipse.

3.3 Detailed Requirements Specification

The System Requirements Specification (SRS) dictates the expected behavior, performance, and constraints of the software.

Functional Requirements:

1. **User Authentication:** The system shall allow administrators to log in using secure credentials to prevent unauthorized access to financial and customer data.
2. **Room Management:** The system shall provide interfaces to add new rooms, update room status (Available, Occupied, Maintenance), and define tariff rates for different room categories.
3. **Guest Registration:** The system must capture essential guest details including Full Name, Contact Number, Government ID Proof, and Address during check-in.
4. **Reservation System:** The system shall allow users to select a check-in and check-out date, verify room availability for those dates, and lock the room to prevent concurrent bookings.
5. **Automated Billing:** Upon checkout, the system must calculate the total cost by multiplying the room rate by the number of nights, adding any extra service charges, and applying standard taxation percentages.

Non-Functional Requirements:

1. **Performance:** The system should process room availability queries in less than 2 seconds to ensure a smooth user experience at the front desk.
2. **Reliability:** The software must handle exceptions gracefully. For example, if a user inputs alphabetical characters in a numeric field (like phone number), the system must not crash but instead display a user-friendly error message.
3. **Usability:** The frontend interface (hotel.html) must be intuitive, requiring less than two hours of training for new hotel staff to become proficient.

CHAPTER 4: SYSTEM DESIGN

4.1 System Architecture

System design is the process of defining the architecture, components, modules, interfaces, and data for a system to satisfy specified requirements. This project utilizes a Client-Server architectural pattern.

- 1. Presentation Layer (Frontend):** Developed using HTML, CSS, and JavaScript. This layer is responsible for rendering the graphical user interface in a web browser. It includes forms for data entry, tables for displaying room availability, and interactive elements for navigation. Client-side validation is implemented using JavaScript to ensure data integrity before it reaches the server.
- 2. Application Logic Layer (Backend):** Developed in Java (`HotelManagement.java`). This layer contains the core business logic. It receives requests from the frontend, processes them (e.g., calculating bills, verifying availability), and manages the temporary data structures. Object-Oriented principles are strictly adhered to. 'Classes' are defined for entities such as Room, Guest, and Booking. 'Inheritance' is used to define specific room types (e.g., `DeluxeRoom` inherits from `Room`). 'Encapsulation' ensures that sensitive data variables are kept private and modified only via authorized getter and setter methods.
- 3. Data Management Layer:** In this prototype version, data is managed in-memory using Java Collections Framework (e.g., `ArrayLists` for lists of rooms, `HashMaps` for mapping booking IDs to guest objects). This ensures rapid data access during runtime.

4.2 Module Description

The application is logically segmented into several highly cohesive and loosely coupled modules:

- **Master Configuration Module:** Accessible only by the hotel manager. Used to initialize the hotel layout (number of floors, rooms per floor) and set the base pricing structure.
- **Front Desk / Reception Module:** The most frequently used module. Provides real-time dashboards showing room statuses color-coded (Green for Available, Red for Occupied, Yellow for Cleaning).

- **Guest Lifecycle Module:** Manages the flow from Guest Check-in (capturing ID, assigning room), to Stay (adding room service charges), to Check-out (freeing the room).
- **Financial / Billing Module:** The computational engine. It reads the duration of stay, fetches the dynamic room tariff, adds service charges, calculates GST/State Tax, and generates a finalized invoice object.

4.3 Data Structures & Schema Design

While a relational database is slated for future scope, the current Java backend simulates a database schema using classes. The primary entities are:

Class Name	Attributes (Variables)	Key Methods
Room	roomNumber (int), type (String), baseRate (double), isOccupied (boolean)	checkStatus(), setOccupied(), getRate()
Guest	guestId (String), name (String), phone (String), idProofType (String)	getDetails(), updateContact()
Booking	bookingId (String), guest (Guest obj), room (Room obj), checkInDate (Date), checkOutDate (Date)	calculateDuration(), generateInvoice()

CHAPTER 5: IMPLEMENTATION

5.1 Overview

The implementation phase translates the system design into executable code. The project is divided into two primary files: the Java backend and the HTML frontend. **Java Backend Implementation (HotelManagement.java):** The backend initializes the room inventory upon startup. It utilizes Scanner classes for console-based testing and can be extended to parse HTTP requests. Core methods include 'checkAvailability()', which iterates through the Room array and checks the boolean 'isOccupied' flag. The 'calculateBill()' method is implemented with strict precision, ensuring floating-point calculations for taxes are accurate. **Frontend Implementation (hotel.html):** The frontend is designed with semantic HTML5 tags for structure. CSS3 is used extensively for styling, utilizing flexbox and grid systems to create a responsive layout that adapts to different screen sizes. A distinct color palette is chosen—calming blues and crisp whites—to reflect the hospitality theme. JavaScript Event Listeners are attached to form submission buttons to intercept the data, validate it (e.g., ensuring check-out date > check-in date), and format it for backend processing.

5.2 Core Code Snippets

Below are representative snippets demonstrating the technical implementation of the system.

Java Backend Logic (Room Class Encapsulation)

```
public class Room {  
    private int roomNumber;  
    private String category;  
    private double nightlyRate;  
    private boolean isAvailable;
```

```

public Room(int roomNumber, String category, double nightlyRate) {
    this.roomNumber = roomNumber;
    this.category = category;
    this.nightlyRate = nightlyRate;
    this.isAvailable = true; // Default state
}

// Getter and Setter Methods for Encapsulation
public boolean checkAvailability() {
    return this.isAvailable;
}

public void bookRoom() {
    if(this.isAvailable) {
        this.isAvailable = false;
    } else {
        throw new IllegalStateException("Room already occupied.");
    }
}

public double getRate() {
    return this.nightlyRate;
}
}

```

Frontend Form Structure (hotel.html)

```

<!-- Guest Check-in Form -->
<div class="booking-container">
    <h2>Guest Check-In</h2>
    <form id="checkInForm" onsubmit="return validateBooking()">
        <label for="guestName">Full Name:</label>
        <input type="text" id="guestName" name="guestName" required>

        <label for="phone">Contact Number:</label>
        <input type="tel" id="phone" name="phone" pattern="[0-9]{10}"

```

required>

<label for="roomType">Select Room Category:</label>

<select id="roomType" name="roomType">

<option value="standard">Standard Non-AC</option>

<option value="deluxe">Deluxe AC</option>

<option value="suite">Executive Suite</option>

</select>

<button type="submit" class="btn-submit">Confirm Booking</button>

</form>

</div>

CHAPTER 6: SYSTEM TESTING

6.1 Testing Methodologies

Software testing is critical to ensure the application is robust, secure, and functions according to the SRS. The following methodologies were employed:

- **Unit Testing:** Individual methods in the Java backend (e.g., the tax calculation algorithm) were tested with various inputs (positive numbers, zero, large floats) to ensure mathematical precision.
- **Integration Testing:** Tested the data flow between the HTML form submission and the Java data structures to ensure strings (like dates) are correctly parsed into Date objects without throwing ParseExceptions.
- **System Validation Testing:** End-to-end testing mimicking a full day of hotel operations (multiple simultaneous check-ins and check-outs) to verify memory stability.

6.2 Comprehensive Test Cases

TC ID	Test Scenario / Description	Test Data / Action	Expected Result	Status
TC_001	Verify Room Availability status update.	Book Room 101 via interface.	Room 101 status changes from 'Green' to 'Red'. Next query shows unavailable.	PASS
TC_002	Attempt double booking.	User 2 attempts to book Room 101 concurrently.	System throws error: "Room 101 is currently	PASS

			occupied." Booking prevented.	
TC_003	Validate Phone Number length constraints.	Enter '12345' in phone field.	HTML5 validation triggers: "Please match the requested format (10 digits)".	PASS
TC_004	Validate Check-out date logic.	Set check-out date BEFORE check-in date.	JavaScript alert prevents submission: "Check-out must be after Check-in".	PASS
TC_005	Verify Base Billing Calculation.	Rate: \$100/night. Stay: 5 nights.	Sub-total calculated precisely as \$500.00.	PASS
TC_006	Verify Tax Implementation.	Sub-total: \$500.00. Apply 18% GST.	Tax calculated as \$90.00. Grand Total displayed as \$590.00.	PASS
TC_007	Empty mandatory fields submission.	Click 'Submit' with empty Guest Name.	Form refuses to submit, highlights Guest Name field in red.	PASS
TC_008	Check-out process functionality.	Initiate check-out for Room 101.	Bill generated, Guest data archived, Room 101 status resets to 'Available'.	PASS

CHAPTER 7: CONCLUSION & FUTURE SCOPE

7.1 Conclusion

The Hotel Management System developed in this project successfully meets its primary objective: to automate and streamline the operational workflow of a hospitality business. By leveraging the robust computational capabilities of Java and the accessible, responsive design principles of HTML/CSS, the application provides a highly effective digital alternative to manual record-keeping. The system successfully mitigates common errors such as double-booking, mathematical billing mistakes, and data loss. Future Enhancements: While the current iteration acts as a powerful functional prototype, several enhancements are planned for future versions to scale the application for enterprise-level deployment: 1. Relational Database Integration: Replacing in-memory Java Collections with a robust RDBMS like MySQL or PostgreSQL to ensure persistent data storage across server reboots. 2. Payment Gateway Integration: Incorporating third-party APIs (e.g., Stripe, PayPal, or Razorpay) to allow guests to make secure online payments directly through the web interface. 3. Mobile Application: Developing a complementary Android/iOS application using frameworks like React Native or Flutter, enabling guests to book rooms and access services directly from their smartphones.

The successful compilation and execution of this project demonstrably prove the efficiency of digital transformation in commercial sectors. By strictly adhering to Object-Oriented design patterns, the software is highly modular, making debugging and future maintenance significantly easier than monolithic legacy systems.

7.2 Future Scope

The foundation laid by this project opens numerous avenues for expansion:

- **Cloud Deployment:** Hosting the application on AWS, Google Cloud, or Azure to allow hotel owners to monitor operations remotely from anywhere in the world.
- **IoT Integration:** Linking the booking system with smart locks on hotel doors, automatically generating and sending digital key codes to the guest's mobile phone upon successful check-in.
- **AI Yield Management:** Implementing machine learning algorithms to analyze historical booking data, local events, and seasonal trends to dynamically adjust room pricing to maximize revenue (similar to airline ticket pricing).
- **Multi-Branch Support:** Upgrading the architecture to a multi-tenant system where a single application instance can manage multiple hotel branches in different geographic locations.

REFERENCES

- Herbert Schildt, "Java: The Complete Reference", 11th Edition, McGraw-Hill Education.
- Jon Duckett, "HTML and CSS: Design and Build Websites", John Wiley & Sons.
- Sommerville, Ian. "Software Engineering", 10th Edition, Pearson.
- Oracle Corporation. "Java SE 8 Documentation." <https://docs.oracle.com/javase/8/docs/>
- Mozilla Developer Network (MDN) Web Docs - HTML5 and CSS3 Specifications.